

INNOPOLIS UNIVERSITY • B22 • COMPUTER SCIENCE

Memory-Assisted LLM as a Judge via Knowledge Graphs for Evaluating Agents.

When does memory help — and how do we trust the measurement?

A U T H O R	Khush Pradipbhai Patel
S U P E R V I S O R	Bader Rasheed
P R O G R A M	09.03.01 — Data Science & AI
D E F E N S E	3 June 2026

What came before.

LLM-AS-A-JUDGE

Automatic evaluators.

MT-Bench · G-Eval · Prometheus.

And the *known biases* they carry — position, length, self-preference.

MEMORY-AUGMENTED

Models with state.

RAG · MemGPT · Reflexion · Generative Agents.

None of them study memory as a *leakage source*.

EVALUATION VALIDITY

Leakage is real.

Balloccu et al. · PoisonedRAG.

Contamination silently *inflates* reported scores.

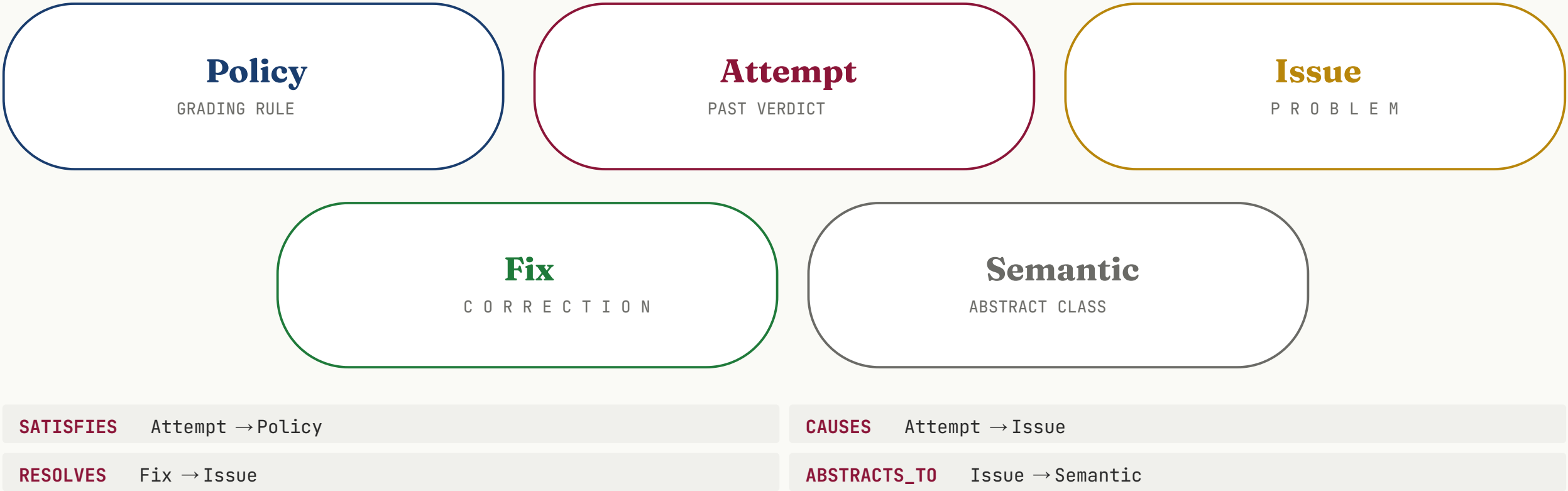
LLMs are now graded by other LLMs.

USE Ranking chatbots. Scoring agents. *Reward signals*

PROBLEM Stateless judges have no consistency mechanism.

IDEA Give the judge a *persistent memory* of past evaluations.

A typed knowledge graph.



An end-to-end example.

INPUT	<i>Task:</i> "Explain key methods for determining the pre-money valuation of a tech startup before a Series A round." <i>Response:</i> a 4-paragraph candidate answer covering DCF, comparable-company, and VC method.
RETRIEVE · 1	Contrastive Attempts · cosine $\geq$ 0.85 <i>Returned:</i> 2 past responses that passed on similar valuation questions, 1 that failed .
RETRIEVE · 2	Similar Issues · cosine $\geq$ 0.90 <i>Returned:</i> "missed discount-rate justification", "vague comparable-company criterion".
RETRIEVE · 3	Semantic Patterns · cosine $\geq$ 0.90 <i>Returned:</i> recurring category — "incomplete-methodology" — observed across 4 past attempts.
ASSEMBLE	All three stages are formatted into a single context block — examples, issues, patterns — and prepended to the judge's prompt alongside the task and rubric.
JUDGE	GPT-4o receives <i>task + rubric + retrieved context + candidate response</i> , and returns a structured verdict (<i>pass/fail</i> + reasoning).

PROPOSED SOLUTION • THE ASSEMBLED PROMPT

What the judge literally receives.

[TASK]

Explain key methods for determining the pre-money valuation of a tech startup before a Series A round.

[RUBRIC]

DCF (future cash flows, projections); Comparable analysis (similar-co. multiples); VC method (rev × multiple - post-\$); founder's share matters; strategic buyers pay more.

[RETRIEVED MEMORY CONTEXT]

IMPORTANT: these are REFERENCE examples only. Each case is UNIQUE. Do NOT assume this case will have the same outcome as similar past cases. Judge THIS response on its own merits against the rubric.

SUCCESSFUL EXAMPLES (similar responses that passed):

- [similarity: 91%] Response excerpt: "DCF projects future cash flows at a stage-appropriate discount rate, while VC method back-solves..."
Why it passed: "covers all three methods, names discount-rate trade-off."
- [similarity: 88%] Response excerpt: "Valuation triangulates DCF + comparable + VC; founder dilution drives the negotiation..."
Why it passed: "explicit on founder share."

FAILED EXAMPLES (similar responses that failed):

- [similarity: 93%] Response excerpt: "Pre-money valuation is the value of the company before investment..."
Why it failed: "tautological; never names a method."

SIMILAR ISSUES OBSERVED:

- "missed discount-rate justification" (sim 0.91)
- "vague comparable-company criterion" (sim 0.90)

RECURRING SEMANTIC PATTERN:

- category: "incomplete-methodology" – seen in 4 prior attempts.

[CANDIDATE RESPONSE]

"Determining the pre-money valuation of a tech startup involves several methods: 1) Discounted Cash Flow (DCF) ... 2) Comparable Analysis ... 3) VC Method ..." (... 4 paragraphs ...)

[INSTRUCTION]

Output a structured verdict: { decision: pass | fail,
reasoning: <short justification> }

Two MCTS extensions.

BUILD #2 • RETRIEVAL LAYER

MCTS-Retrieval

Tree search over **7 retrieval actions** on the graph — four of them *multi-hop traversals* (Issues→Fixes, Semantics→Issues, Policy→Attempts, Attempts→Semantics). Trajectories scored by relevance, diversity, volume.

BUILD #3 • REASONING LAYER

MCTS-Judge

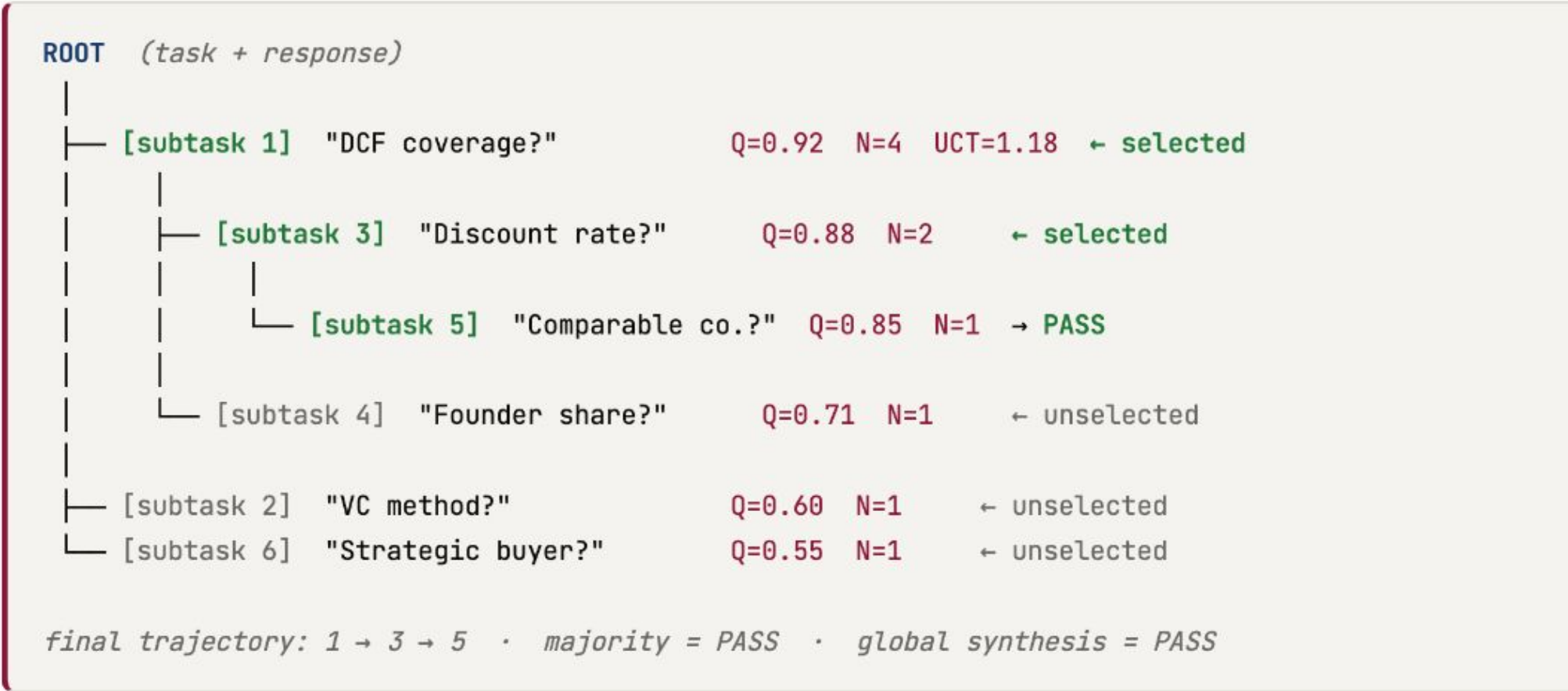
Decomposes evaluation into 5–7 *task-specific subtasks*. Each rollout selects subtasks via UCT + LLM self-assessment. Final verdict aggregates the best trajectory.

Combined into **six evaluation modes**, ablating memory and search independently.

How tree-search reasoning runs.

INPUT	Task + Response. Same EvalsBench item as before.
DECOMPOSE	LLM dynamically generates 5–7 task-specific subtasks — each probes the response from a different angle. <i>e.g. "Does it cover DCF?" · "Is the discount-rate logic correct?" · "Does it mention founder share?"</i>
SEARCH	4 rollouts, max depth 5. At each step, the child node maximising UCT + LLM self-assessment is selected. Each leaf scores the response from that subtask's perspective.
AGGREGATE	Best trajectory's <i>majority vote</i> + a <i>global synthesis</i> over all visited subtasks.
VERDICT	Structured output: <i>{ verdict: pass / fail, reasoning: ... }</i>

One rollout, traced.



UCT balances $Q(s)$ (mean reward) with an exploration bonus that decays as $N(s)$ grows.

And then — a surprise.

MODE	ACCURACY	LATENCY
Stateless	60.0%	3.8 s
MAJ (memory)	63.3%	6.4 s
MCTS-Judge (no memory)	56.7%	126.7 s
MCTS-Judge + Memory	60.0%	51.2 s
MCTS-Retrieval + Judge	60.0%	3.4 s
Full MCTS	46.7%	48.6 s

Early benchmark, 30 samples, un-audited. Reported in the thesis as a *motivating observation*, not a verified result.

But the qualitative picture was clear: something was wrong.

Either the system is broken — or the measurement is.

PROPOSED SOLUTION • PART 3

Can we trust the numbers?

The first numbers were too good.
The next numbers were too strange.
So I went back to the measurement.

Two leakage channels.

CHANNEL I

Paired-item leakage

Question Q

└ Q.PASS → train

└ Q.FAIL → test

The judge sees Q.PASS in memory
while scoring Q.FAIL.

CHANNEL II

Write-back leakage

t=0 score A → write to memory

t=1 score B (related to A)

memory now contains A's verdict

Every early write contaminates
every later read.

Closing both channels.

FIX I Split by *question*, not row.

FIX II *Freeze* memory during the test.

A claim is not a proof. **How do I prove the freeze actually held?**

SHA-256 + runtime write-blocker.

LAYER 1 • AFTER-THE-FACT

SHA-256 fingerprint

```
before = sha256(graph)
... run evaluation ...
after = sha256(graph)
assert before == after
```

LAYER 2 • DURING THE RUN

Runtime write-blocking

```
graph.create = guard
graph.link = guard
graph.upsert = guard
FrozenMemoryViolation if called
```

How the audit runs.

BEFORE	Take a deterministic snapshot : count nodes & edges by type, hash sorted node IDs + edge triples → SHA-256 fingerprint .
DURING	Patch every write method on the graph: <code>create</code> · <code>link</code> · <code>upsert</code> . Any attempted write raises FrozenMemoryViolation .
RUN EVAL	All 80 test items judged. Memory should be read-only.
AFTER	Take another snapshot. Compare to "before".
VERDICT	Pass <i>only if</i> : fingerprints match exactly AND no FrozenMemoryViolation was raised. Otherwise the run is rejected.

Before vs. after — two outcomes.

```
✓ AUDIT PASSES  
nodes: 412 → 412  
edges: 831 → 831  
fingerprint:  
a3f1...8c2b  
== a3f1...8c2b
```

VS.

```
✗ AUDIT FAILS  
nodes: 412 → 492 (+80)  
edges: 831 → 1003 (+172)  
fingerprint:  
a3f1...8c2b  
≠ d9e2...0f17
```

The right side **is** what happened on the first run. The audit forced the bug into the open — and into the fix that followed.

E V A L U A T I O N

Re-evaluation under the audit.

Now — with the audit passing — what do the six modes really do?

EvalBench.

160 samples • finance / startup QA.

80 unique questions — each appearing twice.

50/50 balanced PASS / FAIL.

PAIRED a natural stress test for leakage.

How significance is decided.

Every accuracy is reported with a two-sided Wilson 95% CI. Mode-versus-mode comparisons are paired and assessed with the exact McNemar test.

$$CI_{Wilson} = \frac{p + \frac{z^2}{2n} \pm z\sqrt{\frac{p(1-p)}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \tag{3.4}$$

$$p_{McNemar} = 2 \sum_{k=0}^{\min(b,c)} \binom{b+c}{k} \left(\frac{1}{2}\right)^{b+c} \tag{3.5}$$

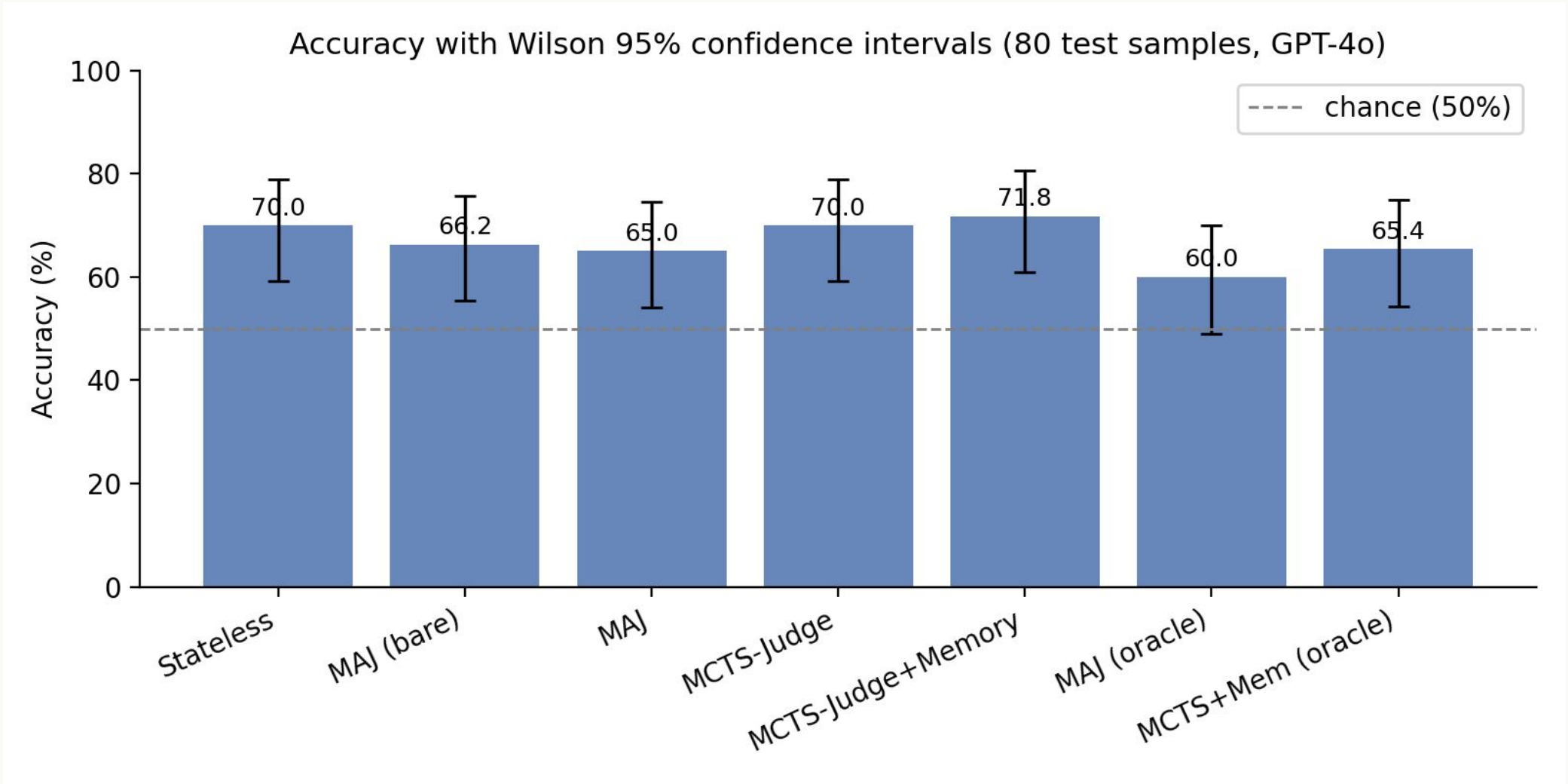
A difference is declared significant only when the McNemar p -value < 0.05 *and* the paired bootstrap CI excludes zero. b, c = discordant pairs.

Under the audited protocol.

MODE	ACCURACY	95% WILSON CI	VS. STATELESS
Stateless	70.0%	[59.2, 78.9]	– baseline –
MAJ (memory)	65.0%	[54.1, 74.5]	-5.0 pp • p = 0.29
MCTS-Judge	70.0%	[59.2, 78.9]	±0.0 pp
MCTS-Judge + Memory	71.8%	[61.0, 80.6]	+1.8 pp • p = 0.80

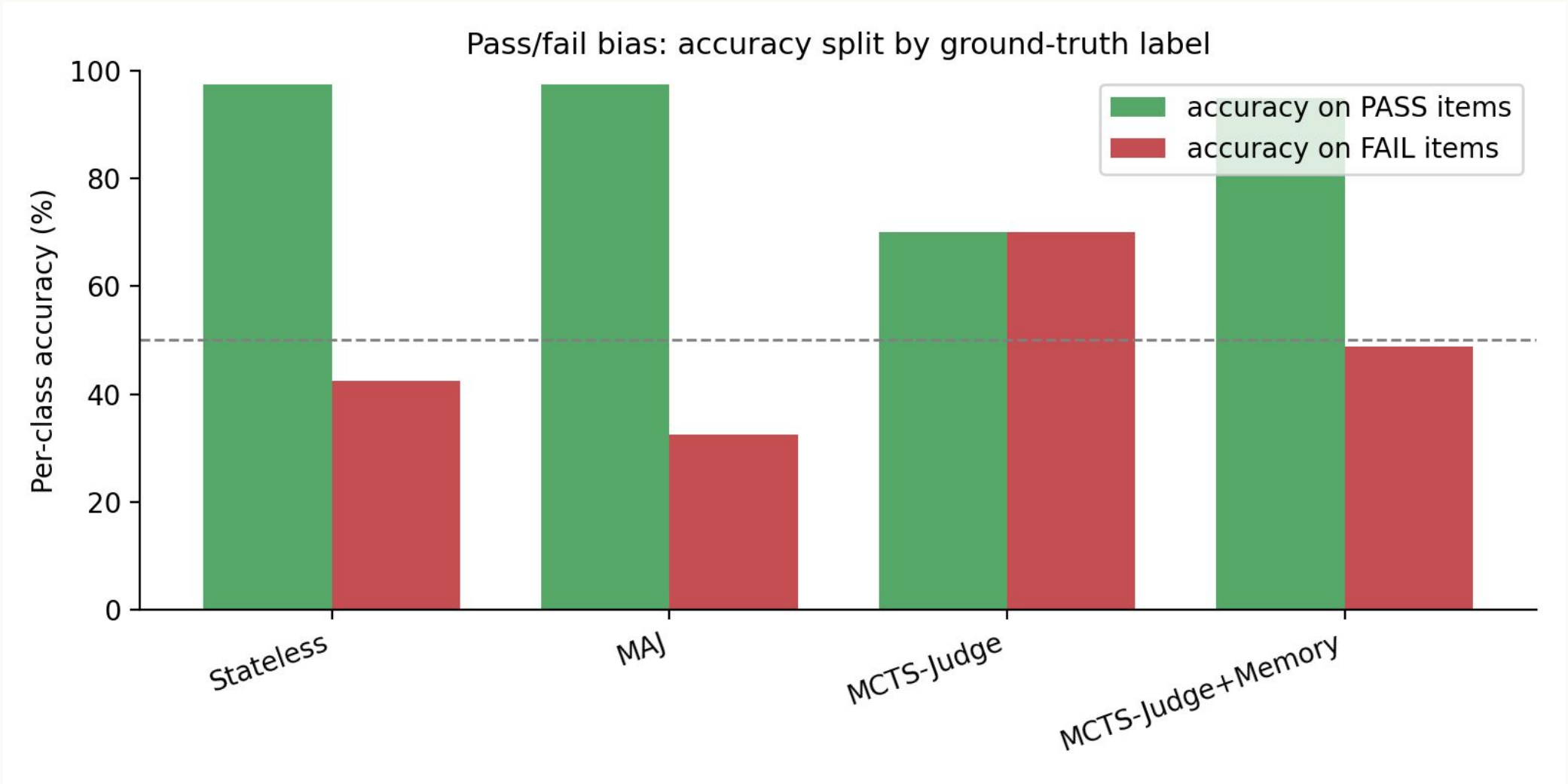
Every CI overlaps the baseline. No paired difference is significant (McNemar & paired bootstrap, $\alpha = 0.05$).

Every error bar overlaps.



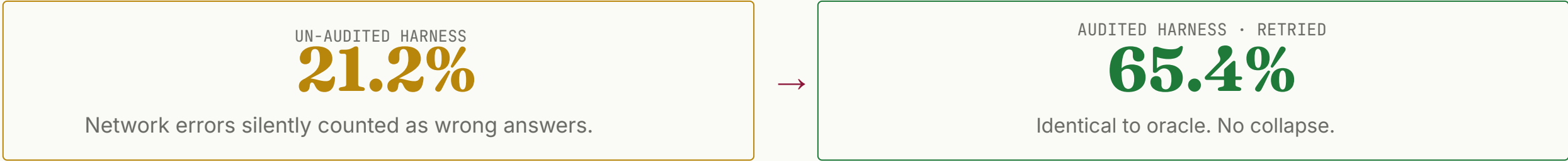
Per-mode accuracy • Wilson 95% CI • n = 80 • GPT-4o

All modes share a pass-oriented bias.



PER-CLASS ACCURACY • GPT-40 • ATTACHING MEMORY DOES NOT CORRECT THE BIAS

What looked like a collapse, wasn't.



A spurious fragility — like a spurious benefit — is a property of the harness, not the system.

Three independent stress tests.

EXPERIMENT B

Stochastic Stability

95% 90%

Same item, judged twice. MAJ and MCTS-Judge+Memory agree with themselves on the great majority of items. **The modes are stable.**

EXPERIMENT C

Label-Flip Discrimination

65% / 70% / 65%

Invert PASS↔FAIL and check the judge flips with it. All modes discriminate above chance — and similarly to each other.

EXPERIMENT D

Defense Filter

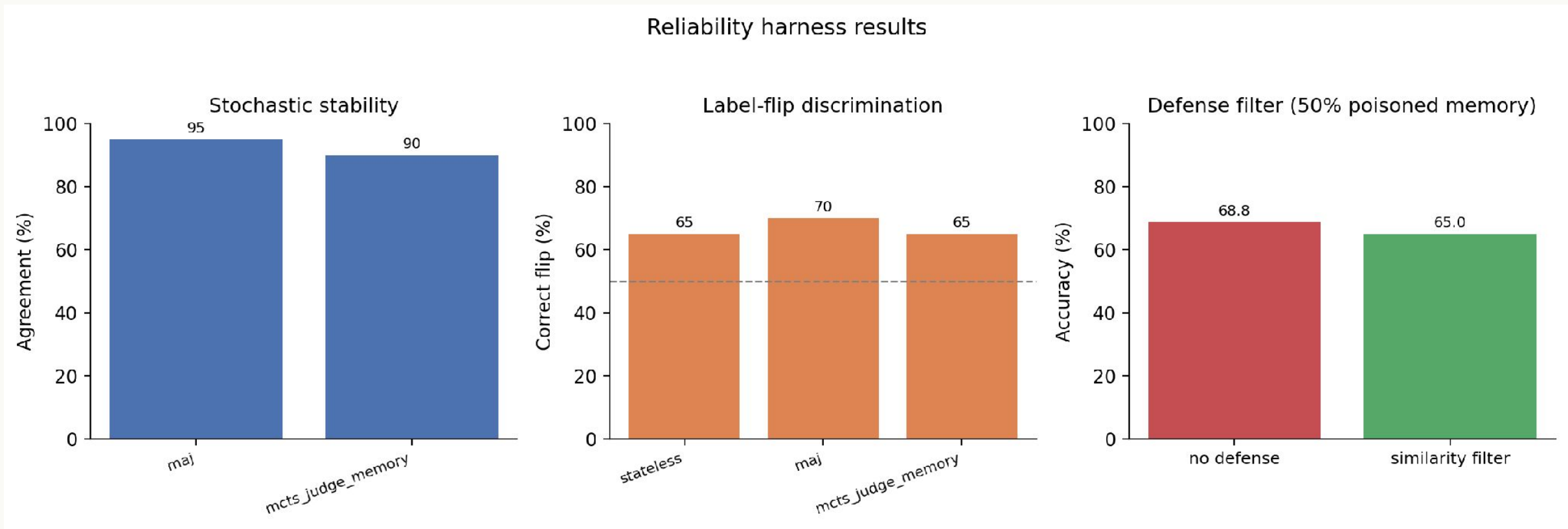
64.8% 65.0%

Similarity-threshold filter under 50% poisoned memory. **The filter does not help.** An honest negative result, reported as such.

How each experiment runs.

STABILITY · B	For each item: judge it twice with the same memory. <i>Measure:</i> agreement rate. "Does the system give the same answer to the same question?"
LABEL-FLIP · C	For each item: generate an <i>inverted</i> response (LLM rewrites it to clearly fail), then judge the inverted version. <i>Measure:</i> how often the verdict correctly flips PASS → FAIL.
DEFENSE · D	Build a 50%-poisoned memory . Compare two configurations: judge with the raw poisoned memory, vs. judge with a <i>similarity-threshold filter</i> applied on retrieved nodes. <i>Measure:</i> does the filter recover accuracy?
VERDICTS	B: 95% / 90% — modes are stable. C: 65% / 70% / 65% — all modes discriminate. D: 64.8% vs 65.0% — the filter does not help.

Stability • discrimination • defense.



THREE ORTHOGONAL RELIABILITY EXPERIMENTS • AUDITED PROTOCOL

FINDINGS • DISCUSSION

Why no help, no harm.

A null result is only credible if it is explained.

Three pieces of evidence.

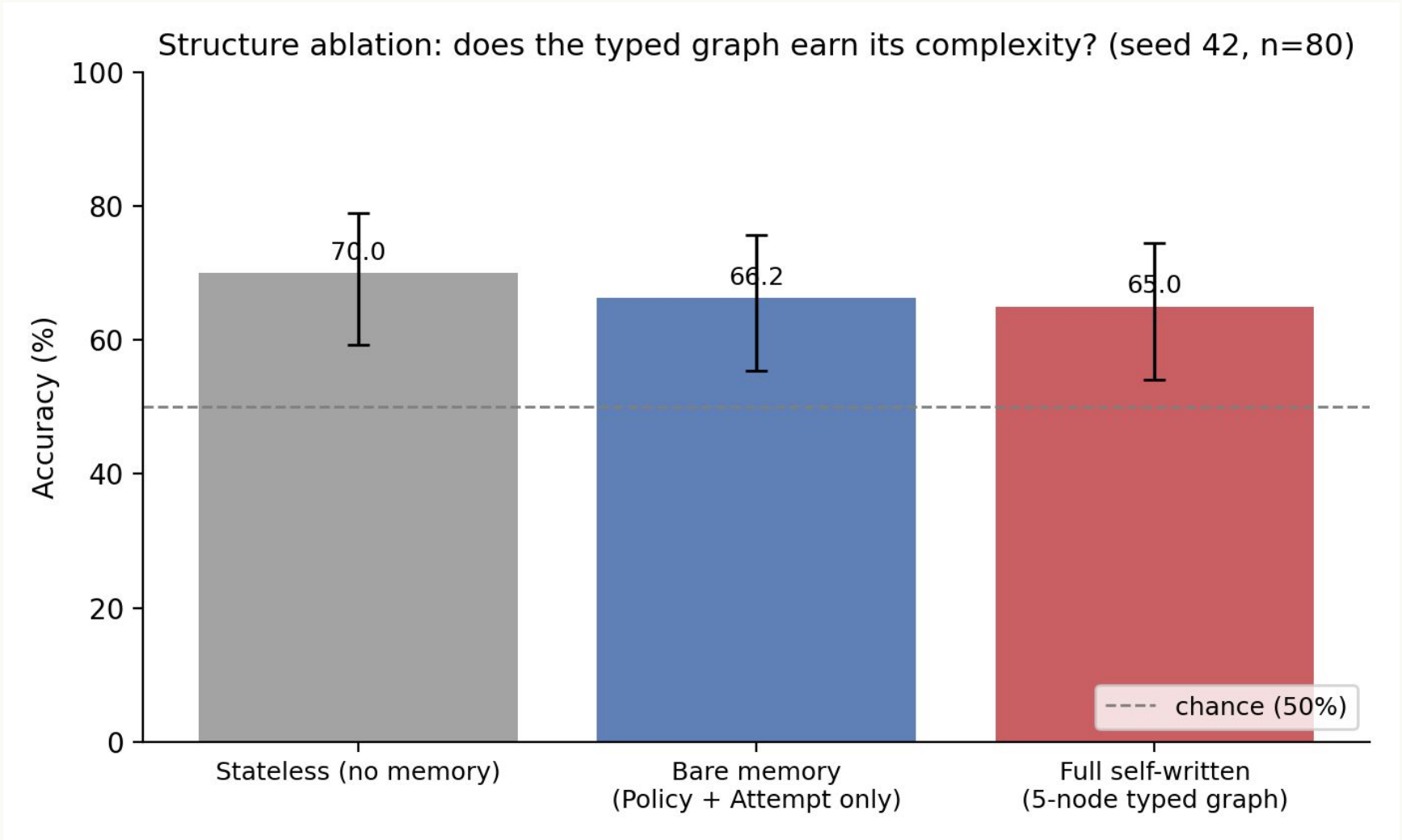
EVIDENCE I Oracle labels don't beat self-written.

EVIDENCE II Bare schema matches the full schema.

HYPOTHESIS GPT-4o sits **near its reasoning ceiling** on this task.

TEST A weaker judge model — future work.

Bare schema = full schema.



STATELESS • BARE MEMORY (POLICY + ATTEMPT) • FULL 5-NODE TYPED GRAPH • WILSON 95% CI

The cost trade-off.

MODE	ACCURACY	MEAN LATENCY	VS. STATELESS
Stateless	70.0%	3.6 s	– baseline –
MAJ (memory)	65.0%	4.5 s	1.3×
MCTS-Judge	70.0%	31.8 s	8.8×
MCTS-Judge + Memory	71.8%	33.3 s	9.2×

MCTS modes cost ~9× more latency for no significant accuracy gain.
Hence: reported as *exploratory*, not recommended.

Where the null holds — and where it doesn't.

SCOPE

One judge model.

All audited results use GPT-4o. A weaker model may have headroom for memory to help; a stronger one may not.

POWER

n = 80, MDE ≈ 12–15 pp.

Powered to detect the conventional 6–10 pp claim, and didn't. A small genuine effect of a few points cannot be excluded — hence "no effect detected at this scale".

DOMAIN

One benchmark, one task.

EvalsBench: rubric-graded QA in finance/startup. Open-ended or creative tasks may behave differently.

WHAT HOLDS

The protocol itself.

The leakage-free audit is benchmark- and model-agnostic. The methodological contribution is independent of the empirical finding.

IN SUMMARY

What this thesis contributes.

Four deliverables.

01 • SYSTEM

MCTS-MAJ

Typed Neo4j memory graph + 3-stage retrieval + two MCTS extensions. Six evaluation modes.

03 • ENGINEERING

Two-layer cryptographic audit

SHA-256 fingerprint + runtime write-blocker. *Proves* the absence of leakage on every run.

02 • METHODOLOGY

Leakage-free protocol

Question-level split + frozen memory. Closes both leakage channels by construction.

04 • FINDING

A corrective null result

Audited protocol overturns the +10 pp gain & the poisoning collapse. Reliability harness shows the system is consistent — but the schema isn't doing extra work.

THE VERDICT

Without the audited protocol,
this thesis would have reported a **+10 pp gain**
and a **catastrophic poisoning collapse**

Both were artifacts.
The audit made the truth
visible.

What comes next.

- Replicate on weaker & stronger judges.
- Scale n by 5–10× (MDE: 13 pp → 4 pp).
- Release the audit as a reusable tool.

BACHELOR'S THESIS DEFENSE • 3 JUNE 2026

Thank you.

Questions are welcome.

CODE <https://github.com/khushpatel1102/maj-research>

BENCHMARK EvalSBench • Khush Patel AI Labs • Apache-2.0